MTS-PolKA: Divisão de tráfego multicaminho em proporção de peso com roteamento na fonte

Giancarlo O. dos Santos, Cristina K. Dominicini, Gilmar L. Vassoler,
Rafael S. Guimarães, Isis Oliveira e Domingos Jose Pereira Paraiso
Programa de Pós-graduação em Computação Aplicada (PPComp)
Instituto Federal do Espírito Santo (Ifes)
Serra, Brasil
{giancarlo, cristina.dominicini, gilmarvassole, rafaelg}@ifes.edu.br,
isisolip@gmail.com, domingos.paraiso@gmail.com

***Resumo*—O artigo apresenta a abordagem *MTS-PolKA* para otimizar o tráfego em redes de data centers com múltiplos caminhos redundantes, visando equilibrar a carga de forma dinâmica e eficiente entre os caminhos para maximizar a largura de banda. A abordagem é baseada em roteamento de fonte multicaminho, permitindo que a fonte faça escolhas de distribuição usando rótulos nos pacotes. Isso possibilita a divisão dinâmica do tráfego sem reconfigurações complexas nas tabelas de comutação. Ao contrário de abordagens anteriores, a *MTS-PolKA* utiliza sistema numérico de resíduos para roteamento de fonte sem armazenamento de estado. Testes demonstram eficácia na distribuição de tráfego e melhorias em relação a abordagens convencionais. *MTS-PolKA* é uma solução inovadora para otimização do tráfego em redes de data centers, pois permite alterar a distribuição de tráfego de forma simultânea em todos os *switches* do caminho.**

***Index Terms*—Redes de data center, engenharia de tráfego, roteamento multicaminho, roteamento na origem, sistema numérico de resíduos, balanceamento de carga, planos de dados programáveis P4, divisão de tráfego.**

I. Introdução

Nos últimos anos, os data centers se tornaram os pilares da infraestrutura de computação. Inúmeros aplicativos de processamento distribuído (como colaboração social, pesquisa, computação de alto desempenho) são executados diariamente em data centers de grande escala que contêm mais de 100.000 servidores [1]. Os data centers modernos são comumente organizados em topologias de árvore multiraiz, como Fat-tree e Clos, para fornecer vários caminhos de custo igual entre diferentes servidores [2].

No entanto, utilizar a largura de banda disponível requer uma maneira eficaz de equilibrar a carga de tráfego, especialmente quando a carga nesses caminhos pode se alterar rapidamente devido a flutuações de tráfego. Portanto, o balanceamento de carga em uma pequena escala de tempo, com base nas condições de tráfego dos caminhos, torna-se crucial para obter um bom desempenho [3]. Assim, para que a arquitetura de rede de *data center* alcance um alto desempenho, a chave é a engenharia de tráfego, a qual o controlador de rede ou os agentes distribuídos devem selecionar caminhos e balancear a carga entre eles para adaptar-se a padrões de tráfego dinâmicos [4].

O tradicional algoritmo de roteamento *ECMP (do inglês Equal Cost Multiple Path)* não atende às necessidades reais devido à sua estratégia de roteamento simples e estática. Nesse contexto, uma ideia comumente adotada é definir, para cada *switch* em um caminho com **n** *switches*, como o tráfego deve ser dividido por pesos para um subconjunto de suas **p** portas. As abordagens existentes que distribuem o tráfego de acordo com os pesos do caminho são baseadas em tabelas *(WCMP [5], do inglês Weighted-Cost MultiPath)* ou estruturas de dados *(DASH [3], do inglês Adaptive Weighted Traffic Splitting in Programmable Data Planes)*. Porém, o problema está na necessidade de alterar os pesos dos caminhos, o que exige modificar as tabelas ou estruturas de dados de cada *switch*.

No caso do *WCMP*, que é baseado em ECMP, é possível explorar vários caminhos, alocar ativamente o fluxo para o caminho apropriado de acordo com a razão de peso e utilizar eficientemente a largura de banda do *link*. Porém, o WCMP utiliza pesos de caminho estáticos, os quais podem não adaptar suas estratégias de divisão às condições do tráfego [3]. Para superar tais limitações do ECMP e WCMP, a proposta do *MP-TCP (do inglês MultiPath TCP)* é dividir cada fluxo em vários subfluxos na camada TCP dos *hosts* e aproveita o ECMP para enviá-los ao longo de vários caminhos. Como o MP-TCP requer mudanças significativas na pilha de protocolo do *host* final, o mesmo pode não ser adequado para todos os ambientes. Especialmente, em plataformas de nuvem pública, em que os clientes, e não os administradores, controlam o sistema operacional e a pilha de rede. Além disso, possui alta complexidade de sinalização e estabelecimento de conexão para fluxos curtos, que normalmente dominam o ambiente do *data center* [1].

Para enfrentar esses desafios, nosso trabalho se baseia na arquitetura M-PolKA (do inglês, Multipath Polynomial Key-based Architecture) [6] para propor uma abordagem de divisão de tráfego multicaminho chamada *MTS-PolKA* (Multipath Traffic Split Polynomial Key-based Architecture), em que é possível selecionar na origem um perfil de

divisão de tráfego em proporção de pesos para cada *switch* do caminho por meio de um rótulo no cabeçalho do pacote. Cada *switch* no caminho mantém uma tabela estática com os perfis de divisão de tráfego disponíveis. A inovação está na capacidade de alterar simultaneamente em cada *switch* do caminho toda a distribuição do tráfego de uma árvore *multicast* com alta granularidade, simplesmente modificando o rótulo no cabeçalho do pacote. Esse método elimina a necessidade de reconfigurar tabelas ou estruturas de dados em cada *switch* e permite uma engenharia de tráfego mais eficiente e dinâmica.

Como prova de conceito, avaliamos a eficácia do inovador método de divisão de tráfego denominado *MTS-PolKA*, através do emulador *Mininet*. Diversos testes em *switches* foram conduzidos, variando os perfis de divisão de tráfego, para demonstrar o funcionamento do *MTS-PolKA*. Os resultados evidenciam a expressividade da nossa proposta para explorar múltiplos caminhos de rede, mantendo a estabilidade do fluxo. Destaca-se ainda a facilidade de transição entre os perfis de divisão de tráfego pelo *MTS-PolKA*, sem a necessidade de complexas configurações nos *switches*, conferindo-lhe vantagem sobre outras abordagens. Em adição, o estudo ressalta a capacidade do *MTS-PolKA* em reduzir o número de estados de rede em comparação com as abordagens tradicionais que dependem de tabelas e outros métodos. Em síntese, o *MTS-PolKA* oferece otimização, adaptabilidade, simplicidade e redução de estados na divisão de tráfego multicaminho, apresentando potencial para aprimorar as redes de data centers.

O restante deste trabalho está organizado da seguinte forma. A Seção 2 apresenta a comparação com os trabalhos relacionados. A Seção 3 descreve o material e métodos propostos. A Seção 4 apresenta os resultados dos experimentos computacionais e a discussão dos resultados. Por fim, a Seção 5 apresenta as considerações finais.

II. Trabalhos Relacionados

Nesta seção, exploramos trabalhos relacionados à divisão de tráfego em redes de Data Center. A divisão de tráfego é vital para equilibrar o fluxo em diferentes caminhos ou servidores [7]. O método mais comum é o *ECMP*, que usa *hashing* para distribuir fluxos igualmente entre caminhos de mesmo tamanho, embora tenha limitações, como colisões e fraquezas em topologias assimétricas [8].

O WCMP generaliza o ECMP para distribuições não uniformes. Utilizando-se de uma estrutura de dados de divisão popular, que replica as entradas da tabela de hash com IDs de caminho em proporção ao peso de cada caminho em uma distribuição atual D, onde D é um conjunto de pesos de caminho. Uma vantagem importante do WCMP é que não há limite no número de caminhos suportados, desde que o tamanho da tabela seja grande o suficiente para manter o erro entre as entradas reais e as entradas desejadas razoavelmente baixo. Mas essa vantagem tem o custo de atualização lenta ou sobrecarga extra. No entanto, atualizar a distribuição D usando

Table I: Tabela comparativa entre diversos esquemas de Engenharia de Tráfego em Redes de Data Center

Método	Granularidade	Local da distribuição de Fluxo	Método de Roteamento	Desvantagens
ECMP	Fluxo	Rede	Tabela Hash	Quando houver colisões na tabela hash incorrerá em fluxos utilizando o mesmo caminho e provável desbalanceamento.
WCMP	Fluxo	Rede	Replica as entradas em Tabela Hash e divisão em proporção de pesos diferentes por caminhos	Atualizar a distribuição requer a modificações dos IDs de caminho para muitas entradas de tabela, o que é inviável devido ao número limitado de acessos à memória por pacote em cada estágio.
HULA	Subfluxo	Rede	Tabela Hash que mantem as informações globais de utilização do link e definir o melhor caminho	Considera apenas o uso de um único caminho "melhor" a qualquer momento.
DASH	Subfluxo	Rede	Mapeamento para diferentes caminhos utilizando o Hash e divisão em proporção de pesos	É uma solução que requer muitas entradas de tabela, o que é inviável devido ao número limitado de acessos à memória por pacote em cada estágio.
MP-TCP - SR	Subfluxo	Servidor na camada de transporte	Roteamento na Origem - Protocolo SR	Requer alterações no host final que podem não ser viáveis em todos os ambientes.
RPS	Pacote	Rede	Random	Pacotes fora de ordem do TCP sob assimetria de rede.
QDAPS	Pacote	Rede	Tabela Hash + Alocação em Fila.	O problema da assimetria de topologia, que muitas vezes é causada por falhas de link e heterogeneidade em equipamentos de rede.
MTS-PolKA	Subfluxo	Rede	Roteamento na Origem – Protocolo M-Polka modificado para Divisão de Tráfego.	O uso de RNS pode exigir a reserva de mais bits no cabeçalho do pacote para transportar os rótulos. No entanto, o M-PolKA, arquitetura que nosso trabalho baseia-se, é projetado para ser escalável e suportar topologias complexas, segundo análise de escalabilidade apresentada [6].

WCMP requer a modificação dos IDs de caminho para muitas entradas de tabela, o que é inviável devido ao número limitado de acessos à memória por pacote em cada estágio [7].

Trabalhos recentes, como *CONGA*, *HULA* e *Contra*, adaptam-se às condições de tráfego no plano de dados. Por exemplo, o *CONGA* usa hardware customizado, enquanto *HULA* e *Contra* usam chaves programáveis em P4, atualizando o tráfego de acordo com as condições, mas limitando-se a um único "melhor" caminho [3].

O artigo *DASH* [3], propõe um esquema de estruturas de dados para divisão de tráfego com reconhecimento de carga no plano de dados. Essas novas estruturas de dados simultaneamente, espalham novos fluxos por vários caminhos em proporção à carga atual nesses caminhos e se ajustam dinamicamente às mudanças de carga nas velocidades do plano de dados [3]. Entretanto, o *DASH* é uma solução que requer muitas entradas de tabela, o que é inviável devido ao número limitado de acessos à memória por pacote em cada estágio. E para atualização a cada entrada da tabela, precisamos de muitos acessos aos registradores.

Uma solução proposta recentemente chamada MP-TCP propõe uma divisão de tráfego em uma granularidade de subfluxo. O primeiro esforço, MP-TCP, divide um fluxo TCP em vários subfluxos nos hosts finais, que são roteados por diferentes caminhos na rede usando ECMP. O *host* final receptor agrega os subfluxos TCP e reordena os pacotes [9]. O problema é que o *MP-TCP* requer alterações no *host* final que podem não ser viáveis em todos os ambientes. Ele também sofre de alta sobrecarga para fluxos curtos que dominam os *data centers* [1].

Os trabalhos citados anteriormente são baseados em rede definida por software (SDN, pela sigla em inglês) para redes data centers sendo eficazes e práticos. Embora seja prática, a técnica SDN apresenta limitações, como a necessidade de criar regras de encaminhamento individuais para cada fluxo, resultando em tabelas de fluxo crescentes que consomem recursos e energia. Isso é agravado pelo uso de memória TCAM (do inglês, Ternary Content Addressable Memory), o que é caro e consome energia [9].

Algumas abordagens como o *random packet spraying*

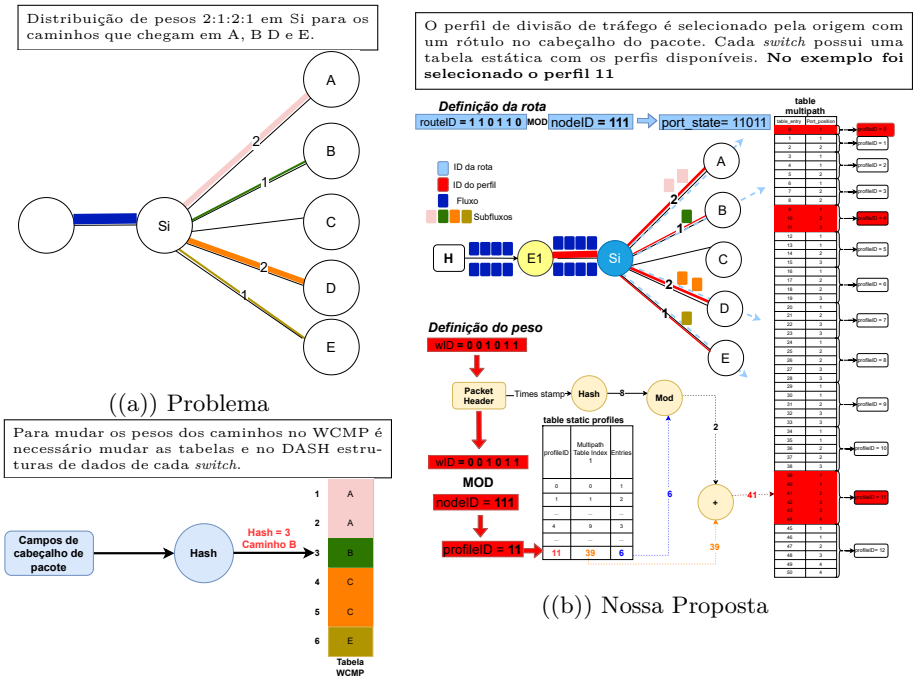


Figure 1: Ilustração da Proposta

(RPS) e o*Queuing Delay Aware Packet Spraying (QDAPS)* são propostas para mitigar a reordenação de pacotes em fluxos, mas enfrentam problemas com pacotes TCP desordenados devido a assimetrias de rede e topologia. A assimetria pode ser causada por falhas de link e heterogeneidade de equipamentos, exigindo ajustes na seleção de caminhos [10].

Neste artigo, é abordada a otimização da alocação de tráfego em redes de data center, com foco na eficiência e adaptabilidade. A abordagem apresentada permite que as origens do tráfego escolham perfis de divisão via rótulos nos pacotes, eliminando reconfigurações complexas nos switches. A estratégia também emprega roteamento multicaminho para melhorar o desempenho da rede, maximizar a largura de banda e reduzir estados de rede, comparado a abordagens convencionais com tabelas, como destacado na tabela I.

III. Proposta

Neste trabalho, apresentamos uma abordagem inovadora com o objetivo de otimizar a divisão de tráfego em um caminho composto por **n** *switches*. Cada *switch* deve ser configurado para distribuir o tráfego entre um subconjunto de suas **p** portas, considerando os pesos atribuídos a essas portas, conforme ilustrado na Figura 1(a).

Nossa abordagem tem como objetivo superar as limitações das abordagens existentes, como o uso de tabelas estáticas *WCMP* ou estruturas de dados complexas *DASH*, que frequentemente exigem reconfigurações nos *switches* para ajustar os pesos dos caminhos, conforme evidenciado na Figura 1(c). Em contraste, nossa inovação permite que a origem do tráfego selecione a divisão de tráfego usando rótulos no cabeçalho do pacote. Cada *switch* mantém uma tabela estática com perfis de divisão disponíveis. Isso proporciona alta granularidade e flexibilidade para ajustar a divisão de tráfego em tempo real, sem a necessidade de reconfigurações complexas nos *switches*, resultando em melhor desempenho e eficiência nas redes de data centers com múltiplos caminhos redundantes, conforme representado na Figura 1(b).

A base da nossa proposta é o roteamento de fonte em redes, usando uma extensão do protocolo *M-PolKA* [6], que explora o sistema numérico de resíduos (RNS, do inglês, *Residue Number System*) [11]. Esse sistema oferece um roteamento de fonte sem armazenamento de estado, onde as decisões de encaminhamento nos nós centrais dependem de operações de módulo sobre um rótulo de rota. Essa arquitetura envolve nós de núcleo, de borda e um controlador centralizado, permitindo o roteamento multicaminho por meio de identificadores polinomiais em GF(2): *routeID*, *nodeID* e *portID*. Isso viabiliza o roteamento de fonte sem armazenamento, utilizando operações de módulo entre *routeID* e *nodeID* para decisões de encaminhamento e obtenção do *portID* correspondente. Essa abordagem possibilita o roteamento sem armazenamento de estado, baseando-se em operações simples de módulo em rótulos de rota para definir as portas de saída [6].

A proposta do artigo *M-PolKA* clona os pacotes por múltiplos caminhos. De forma diferente, este trabalho propõe a aplicação do protocolo *M-PolKA* para dividir o

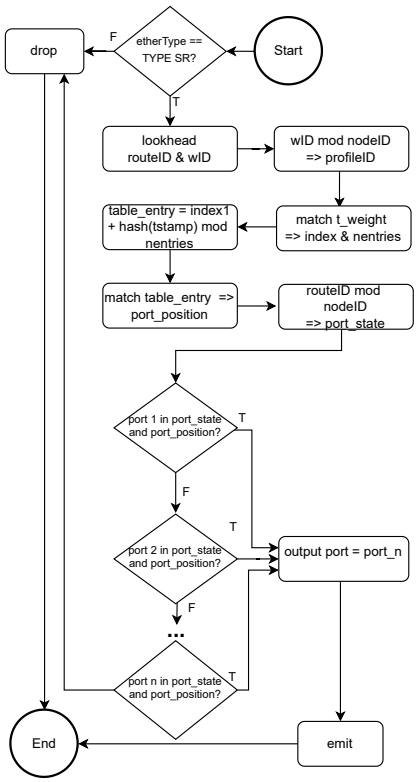


Figure 2: Pipeline do *MTS-PolKA*

tráfego com proporção de peso entre as diferentes portas de saída. Isso permite explorar a diversidade de multicaminho para fornecer balanceamento de carga. Diferentes perfis de peso são aplicados à divisão de tráfego para realizar uma distribuição eficiente em redes de data center.

Nossa arquitetura é composta por nós do núcleo da rede (em azul), nós de borda (em amarelo) e um controlador lógico centralizado. O roteamento de fonte multi caminho utiliza cinco identificadores polinomiais sobre GF(2): (i) *routeID*, um identificador de rota multi caminho calculado pelo controlador usando o Teorema Chinês do Resto (CRT) [11] e inserido no pacote pelos nós de borda; (ii) *nodeID*, um identificador atribuído aos nós do núcleo durante a fase de configuração da rede; (iii) *port_state*, um identificador atribuído às portas de saída de cada nó de núcleo; (iv) *wID*, um identificador de peso que realiza a divisão do tráfego com base nos perfis da Figura 1(b), também calculado pelo controlador usando o CRT e inserido no pacote pelos nós de borda; e (v) *profileID*, um identificador obtido pela operação de módulo binário entre o *wID* do pacote e o *nodeID*.

Em nossa implementação, cada *switch* de borda possui uma tabela preenchida pelo controlador, que insere os rótulos *routeID* e *wID*. Essa tabela mapeia informações sobre o fluxo (e.g., endereço IP de destino) em decisões de roteamento e balanceamento de carga. Posteriormente, esses rótulos são usados por cada nó de núcleo para determinar o estado das portas de saída e perfis de divisão de tráfego correspondentes. Cada nó de núcleo possui duas tabelas estáticas com perfis de tráfego pré-definidos, que são selecionados a partir de operações com os rótulos *routeID* e *wID* de cada pacote. Dessa forma, a borda da rede pode configurar perfis de tráfego distintos em cada *switch* de núcleo para cada pacote sem que seja necessária nenhuma configuração adicional nos *switches* de núcleo.

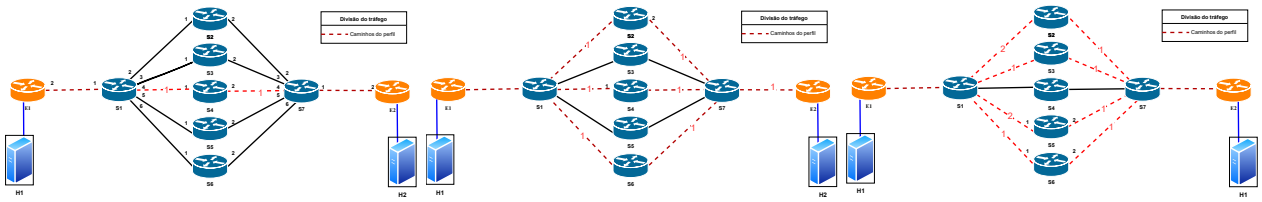
Conforme exemplo ilustrado na Figura 1(b), o *switch* de núcleo Si realiza o roteamento usando a operação de módulo entre *routeID* = 110110 e *nodeID* = 111 para calcular a rota e obter as portas de saída ativas (*port_state* = 11011). Dessa forma, o tráfego é encaminhado pelas portas correspondentes aos bits de valor 1 em *port_state*, com a análise realizada da direita para a esquerda. Depois, ainda é necessário descobrir qual porcentagem do tráfego será encaminhado para cada porta de saída ativa. Ao empregar a operação de módulo entre $wID = 001011$ e $nodeID = 111$, o *switch* Si do núcleo da rede obtém *profileID* = 11, que é o perfil de divisão de tráfego escolhido. Com base na tabela *table static profiles*, para *profileID* = 11, existem 6 entradas a partir da posição 39. Essa informação é usada para acessar a tabela *table multipath*, em que o perfil escolhido usa peso 2:1:2:1 respectivamente para a primeira, segunda, terceira e quarta porta ativas.

Para explorar os pesos definidos na tabela *table multipath* e espalhar o tráfego de forma adequada, o *switch* Si executa uma função de *hash* usando o *timestamp*. O resultado do *hashing* para este exemplo, que é 8, é submetido à operação de módulo pelo número de entradas, que é 6 com resultado 2. Esse valor é adicionado ao índice 39 da *table static profiles*, resultando no índice 41 da *table multipath*. Por fim, esse valor corresponde à entrada da tabela que define a posição da porta de saída `port_position` de valor 2. Isso significa que a segunda porta ativa deve ser escolhida como porta de saída para este pacote do exemplo de acordo com a variável *port_state* = 11011 calculada a partir do *routeID*. Dessa forma, aplicando o espalhamento descrito, o tráfego dividido de acordo com o perfil *profileID* = 11 será direcionado para as primeira, segunda, terceira e quarta portas ativas, com uma proporção de peso de $2:1:2:1$, respectivamente.

No núcleo da rede, o pipeline de plano de dados representado na Figura 2 foi implementado usando a linguagem P4. Quando o *etherType* é *TYPE SR*, o *MTS-PolKA* lê seletivamente cabeçalhos de pacotes via *lookahead* P4, avaliando um conjunto de bits sem mover o índice. Para definir os pesos das portas de saída, o *switch* executa módulo entre *wID* e *nodeID*, além de módulo entre *routeID* e *nodeID*, para determinar proporções e estados de transmissão. Isso permite calcular pesos e estados das portas de transmissão de maneira eficiente.

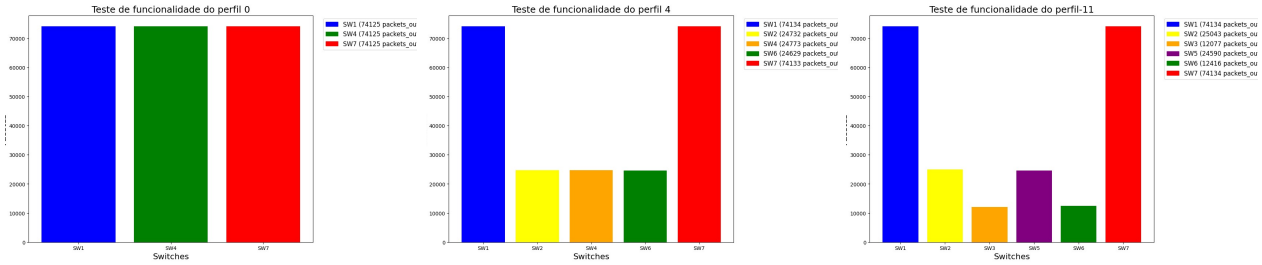
IV. Avaliação

Nesta seção, descrevemos os passos da pesquisa, os materiais utilizados, os métodos empregados e as métricas de avaliação. Utilizamos o emulador *Mininet* para conduzir



((a)) Teste funcional do perfil 0 ((b)) Teste funcional do perfil 4 ((c)) Teste funcional do perfil 11

Figure 3: Testes funcionais dos perfis



((a)) Teste funcional do perfil 0 ((b)) Teste funcional do perfil 4 ((c)) Teste funcional do perfil 11

Figure 4: Resultado dos Testes funcionais dos perfis

experimentos especiais, como compilação de programas P4-16 e configuração de *switches bmv2*. O ambiente de desenvolvimento e testes foi o emulador *Mininet*, usando um servidor *PowerEdge* R650, com 32 GB de *RAM* e processador Intel *Xeon Silver* 4314 de 2.4 GHz. Avaliamos o desempenho do *MTS-PolKA* por meio de dois experimentos, a fim de testar as funcionalidades dos perfis de divisão de tráfego. Isso inclui a seleção e reconfiguração ágil de perfis de divisão de tráfego na origem, eliminando a necessidade de reconfigurações complexas nos *switches*.

A. Testes funcionais dos perfis de divisão de tráfego

Para os testes realizados, três diferentes perfis de tráfego (0, 4 e 11) foram aplicados, cada um com diferentes proporções de distribuição de tráfego nas portas ativas, com base em três cenários (ilustrados na figura 3). A divisão de tráfego foi determinada pela origem, o *switch* E1, usando rótulos no cabeçalho do pacote. Os valores de *routeID* e *wID* variaram de acordo com o perfil.

Nesses testes, os perfis 0, 4 e 11 foram configurados no *switch* S1, enquanto os demais *switches* mantiveram uma distribuição de tráfego constante usando o perfil 0, direcionando todo o tráfego pela primeira porta ativa. O teste envolveu medições de pacotes usando a ferramenta *bwm-ng* para exibir as quantidades de tráfego. O tráfego foi gerado via *iperf3*, com host2 como servidor e *host1* enviando pacotes UDP com vazão de 10 Mbps.

Teste com Perfil 0: Neste teste, usando uma proporção de peso de 1 na primeira porta ativa, todo o tráfego de saída foi roteado para a porta `s1-eth4` no *switch* `S1` (conforme mostrado na Figura 3(a)). Outros *switches* também direcionaram o tráfego através de suas primeiras portas ativas. Os resultados (Figura 4(a)) confirmaram a divisão adequada do pacote UDP de acordo com o perfil 0 nos *switches* `SW1`, `SW4` e `SW7`.

Teste com Perfil 4: Neste teste, empregando uma proporção de peso de 1:1:1 (como na Figura 3(b)), o *switch* `SW1` dividiu o tráfego de saída uniformemente em suas três portas ativas (`s1-eth2`, `s1-eth4` e `s1-eth6`). Outros *switches* enviaram tráfego por meio de suas primeiras portas ativas. A Figura 4(b) apresenta os resultados, ilustrando a distribuição de pacotes UDP seguindo o perfil 4 (1:1:1) nos *switch* `SW1`, `SW2`, `SW4`, `SW6` e `SW7`.

Teste com o Perfil 11: Para este teste, utilizando uma proporção de peso de 2:1:2:1 (conforme mostrado na Figura 3(c)), o *switch* `SW1` distribuiu o tráfego de forma variada em suas quatro portas ativas. Outros *switches* mantiveram um peso 1 em suas primeiras portas ativas. Os resultados, representados na Figura 4(c), demonstraram a distribuição de pacotes UDP alinhada com o perfil 11 (2:1:2:1) nos *switches* `SW1`, `SW2`, `SW4`, `SW6` e `SW7`.

B. Migração ágil de perfis de divisão de tráfego

O experimento consistiu na seleção ágil de perfis de divisão de tráfego na origem, onde a rede trocou entre os perfis 0, 4 e 11, conforme figura 3. A divisão de tráfego foi determinada pela origem, o *switch* E1, usando rótulos no cabeçalho do pacote. Os valores de *routeID* e *wID* variaram de acordo com o perfil. O teste ocorreu em três fases: perfil 0 por 10 segundos, perfil 4 por 10 segundos e perfil 11 por 10 segundos.

Foram conduzidos 30 testes nos quais as médias de vazão foram registradas. As médias de vazão foram apresentadas na figura 5, evidenciando um fluxo estável durante as

transições entre os perfis. A abordagem adotada incluiu rótulos nos cabeçalhos dos pacotes (*wID* e *routeID*) e o uso de tabelas estáticas nos *switches*. Essa abordagem permitiu selecionar os perfis de divisão de tráfego com base na origem, proporcionando flexibilidade e granularidade na distribuição de tráfego. Dessa forma, demonstramos que *MTS-PolKA* resolve o problema de reconfiguração ágil de perfis de divisão de tráfego. Em abordagens baseadas em tabelas, há uma latência grande para configuração e convergência, já que o controlador precisa aplicar as alterações a todos os nós afetados ao longo do caminho. Com *MTS-PolKA*, a divisão de tráfego pode ser reconfigurada rapidamente apenas definindo novos rótulos na borda sem necessidade de atualizar as tabelas dos nós de núcleo.

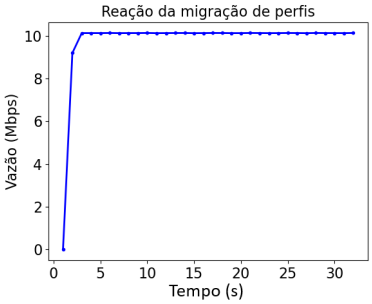


Figure 5: Resultado do Teste de troca de perfis

Comparando com o *WCMP*, que replica entradas de tabela com *IDs* de caminho baseados em pesos de caminho, nossa abordagem é vantajosa, pois o *WCMP* enfrenta atualizações lentas e sobrecarga adicional. Trabalhos como *CONGA*, *HULA* e *Contra* atualizam o estado do plano de dados para direcionar o tráfego com base nas condições de tráfego, mas estão limitados a considerar um único caminho "melhor" de cada vez. O método *DASH* cria e mantém um *pipeline* sensível à carga para a divisão de tráfego, lidando com situações complexas de tráfego, mas cada etapa desse *pipeline* exige cálculos e operações específicas, como cálculos de fronteiras de hash e armazenamento nas memórias locais, resultando em desenvolvimento e gerenciamento detalhados.

Conclusão

Neste artigo, introduzimos uma abordagem inovadora denominada *MTS-PolKA*. Essa técnica permite que, na origem, se selecione um perfil de divisão de tráfego baseado em proporção de pesos para cada *switch*. Diferentemente das abordagens tradicionais, como *ECMP* e *WCMP*, que enfrentam limitações consideráveis na adaptação a flutuações de tráfego, *MTS-PolKA* tem como principal inovação a capacidade de alterar, com alta granularidade, a distribuição de tráfego de forma simultânea em todos os *switches* do caminho de uma árvore *multicast*. Tais alterações são efetuadas simplesmente através da modificação do rótulo presente no cabeçalho do pacote.

Em nossos experimentos, observamos alta granularidade e flexibilidade na divisão de tráfego, sem requerer reconfigurações complexas nos *switches*. Isso resultou em aprimoramento no desempenho e eficiência das redes de data centers, uma vez que otimiza o uso dos caminhos redundantes. Adicionalmente, notamos uma rápida reconfigurabilidade do tráfego, uma vez que a reconfiguração se concentra exclusivamente na origem.

Em resumo, *MTS-PolKA* emerge como uma solução promissora para engenharia de tráfego em redes de data centers, apresentando-se como uma ferramenta poderosa para enfrentar os desafios da distribuição de tráfego de forma que se consiga aproveitar o multicaminhos que uma rede de data center oferece. Trabalhos futuros incluem extensão da avaliação de desempenho da proposta e comparação com os trabalhos relacionados.

References

[1] A. Dixit, P. Prakash, Y. C. Hu, and R. R. Kompella, "On the impact of packet spraying in data center networks," in *2013 Proceedings IEEE INFOCOM*, 2013, pp. 2130–2138.

[2] J. Huang, W. Lyu, W. Li, J. Wang, and T. He, "Mitigating packet reordering for random packet spraying in data center networks," *IEEE/ACM Transactions on Networking*, vol. 29, no. 3, pp. 1183–1196, 2021.

[3] K.-F. Hsu, P. Tammana, R. Beckett, A. Chen, J. Rexford, and D. Walker, "Adaptive weighted traffic splitting in programmable data planes," in *Proceedings of the Symposium on SDN Research*, ser. SOSR '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 103–109. [Online]. Available: https://doi.org/10.1145/3373360.3380841

[4] S. A. Jyothi, M. Dong, and P. B. Godfrey, "Towards a flexible data center fabric with source routing," in *Proceedings of the 1st ACM SIGCOMM Symposium on Software Defined Networking Research*, 2015, pp. 1–8.

[5] Z. Cui, Y. Hu, and S. Hou, "Adaptive weighted cost multipath routing on pisa," in *2021 IEEE International Conference on Artificial Intelligence and Industrial Design (AIID)*, 2021, pp. 541–544.

[6] R. S. Guimarães *et al.*, "M-polka: Multipath polynomial key-based source routing for reliable communications," *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2639–2651, 2022.

[7] O. Rottenstreich, Y. Kanizo, H. Kaplan, and J. Rexford, "Accurate traffic splitting on commodity switches," in *Proceedings of the 30th on Symposium on Parallelism in Algorithms and Architectures*, ser. SPAA '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 311–320. [Online]. Available: https://doi.org/10.1145/3210377.3210412

[8] R. Valentim, C. Dominicini, D. Mafioletti, R. Villaca, M. Ribeiro, and M. Martinello, "Rdna balance: Balanceamento de carga por isolamento de fluxos elefante em data centers com roteamento na origem," in *Anais do XXXVII Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos*. Porto Alegre, RS, Brasil: SBC, 2019, pp. 1000–1013. [Online]. Available: https://sol.sbc.org.br/index.php/sbrc/article/view/7418

[9] J. Pang, G. Xu, and X. Fu, "Sdn-based data center networking with collaboration of multipath tcp and segment routing," *IEEE Access*, vol. 5, pp. 9764–9773, 2017.

[10] Y. Zhang, J. Bi, Z. Li, Y. Zhou, and Y. Wang, "Vms: Load balancing based on the virtual switch layer in datacenter networks," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 6, pp. 1176–1190, 2020.

[11] C. Dominicini *et al.*, "Polka: Polynomial key-based architecture for source routing in network fabrics," in *2020 6th IEEE Conference on Network Softwarization (NetSoft)*, 2020, pp. 326–334.